



SmartStock

Project Engineering

Year 4

Anesu Njove

Bachelor of Engineering (Honours) in Software and
Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

_____ Anesu Njove _____

Acknowledgements

I would like to thank my project supervisor, Fergus Downey, for his guidance and support throughout this project. I would also like to thank the staff of the Software and Electronic Engineering department at Atlantic Technological University for the knowledge and skills they have imparted over the course of this programme.

Table of Contents

Summary	5
1 Poster.....	6
2 Introduction	7
3 Background	8
3.1 Inventory Management Systems.....	8
3.2 Time-Series Forecasting with Prophet	8
3.3 Random Forest Classification	9
3.4 Microservices Architecture.....	9
4 Project Architecture	10
4.1 Authentication Service	11
4.2 Inventory Service.....	11
4.3 Forecasting Service	11
4.4 Frontend	11
5 Project Plan.....	12
6 Implementation.....	13
6.1 Database Design	13
6.1.1 Inventory Database.....	13
6.1.2 Authentication Database	16
6.1.3 Containerisation of Database	17
6.2 Demand Forecasting	18
6.2.1 Data Retrieval	18
6.2.2 Data Preparation.....	18
6.2.3 Model Fitting and Prediction.....	19
6.2.4 Forecast Visualisation	20
6.3 Stockout Risk Prediction – Research	21
6.4 Barcode Integration	27

6.5	Deployment	31
7	Testing	33
8	A.I Usage	37
9	Conclusion	41
10	Appendix	42
11	References	43

Summary

SmartStock is an AI-powered inventory and demand forecasting system designed for use in pharmaceutical and warehouse stock management environments. The system addresses a common problem in these settings: the difficulty of accurately predicting future stock demand and identifying items at risk of running out before they can be replenished.

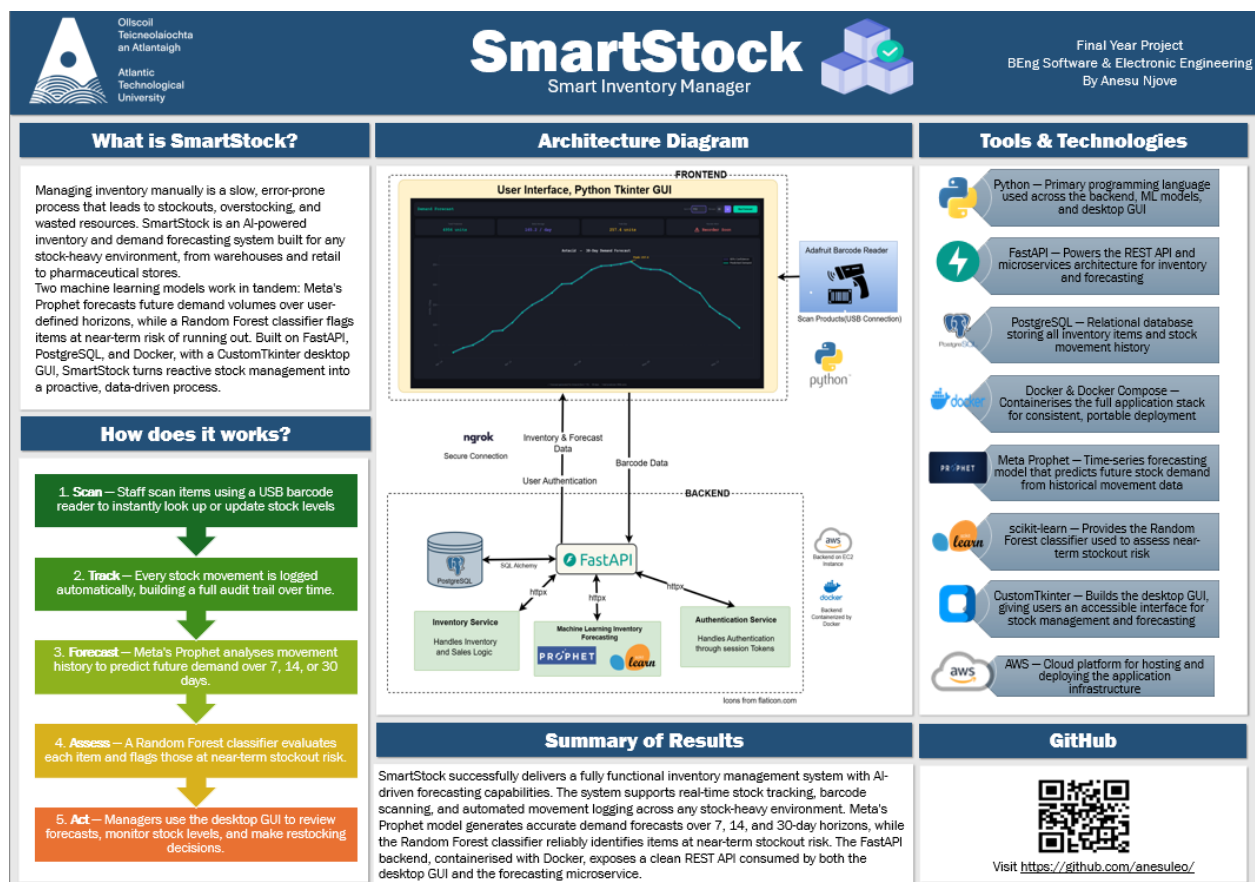
The project was implemented as a microservices architecture comprising four independently deployable services: an Authentication Service, an Inventory Service, a Forecasting Service, and a Desktop GUI Frontend. Each service is containerized using Docker and published to GitHub Container Registry, with a dedicated deployment repository allowing all services to be launched on any machine with a single command.

The core AI component is Meta Prophet, a statistical time-series forecasting library used to predict future demand based on historical sales movements. Given historical OUT movements for any inventory item, the system generates a forecast of daily demand over a configurable horizon with confidence intervals, peak demand annotation, and a reorder alert. As part of the research phase, the use of a scikit-learn Random Forest classifier for stockout risk prediction was also investigated as a potential complement to the Prophet forecasting model, representing an area for future implementation.

The project also integrates barcode scanning hardware, a PostgreSQL database backend, a CI/CD pipeline using GitHub Actions, and deployment to an AWS EC2 instance.

Comprehensive test suites were written for both the Authentication and Inventory services using pytest, with all tests running automatically on every push to the main branch.

1 Poster



2 Introduction

Effective inventory management is critical in pharmaceutical and warehouse settings. Overstocking ties up capital and risks product expiry, while understocking can interrupt the supply of essential medications. Despite this, many smaller pharmacies and warehouses still rely on manual processes or basic spreadsheet-based systems that offer no predictive capability.

SmartStock was developed to address this gap. The goal of the project was to build a full-stack, AI-powered inventory system that not only tracks stock levels but actively forecasts future demand and alerts staff to potential shortages before they occur. The system was designed with real-world deployment in mind, incorporating containerization, cloud deployment, and hardware integration.

This report documents the design, implementation, and testing of the SmartStock system. It covers the system architecture, the technologies chosen and the rationale behind those choices, the implementation of the AI components, the CI/CD pipeline, hardware integration, and the results of testing. The report also reflects on challenges encountered during development and decisions made in response to those challenges.

3 Background

3.1 Inventory Management Systems

An inventory management system (IMS) is a software tool used to track the quantity, location, and movement of stock items. Traditional IMS solutions record stock levels and movements but do not provide predictive capabilities. Modern systems increasingly incorporate machine learning to forecast demand, identify trends, and automate reordering decisions [1].

In pharmaceutical settings, IMS solutions must also account for product expiry dates, regulatory compliance, and the critical nature of stock availability. A stockout of a required medication can have serious consequences, making demand forecasting particularly valuable in this domain [2].

3.2 Time-Series Forecasting with Prophet

Meta Prophet is an open-source time-series forecasting library developed by Meta. It is designed to handle datasets with strong seasonal patterns, missing data, and irregular trends. Prophet uses a decomposable model with three main components: trend, seasonality, and holidays. It fits a piecewise linear or logistic growth curve to the trend component and uses Fourier series to model seasonality [3].

Prophet was chosen for SmartStock because of its robustness to missing data and its ability to produce interpretable forecasts without requiring extensive hyperparameter tuning. Unlike ARIMA models, which require the data to be stationery and struggle with missing values, Prophet handles gaps in sales history gracefully — a common occurrence in pharmacy sales data.

3.3 Random Forest Classification

A Random Forest is an ensemble machine learning method that constructs multiple decision trees during training and outputs the class that is the mode of the classes predicted by individual trees [4]. It is robust to overfitting and performs well on tabular data with a relatively small number of features.

In SmartStock, a Random Forest classifier is used to predict which inventory items are at risk of stocking out within a 30-day window. This is a binary classification problem: given the current stock level, recent sales rate, and days until expiry, will this item stock out in the next 30 days? This is a fundamentally different question from the one Prophet answers — Prophet forecasts volume over time, while the Random Forest flags immediate risk.

3.4 Microservices Architecture

A microservices architecture decomposes an application into a set of small, independently deployable services, each responsible for a specific business capability and communicating over well-defined APIs [5]. This contrasts with a monolithic architecture where all functionality is packaged into a single deployable unit.

4 Project Architecture

SmartStock is built as a microservices system comprising four components, each running as an independent process communicating via HTTP REST APIs.

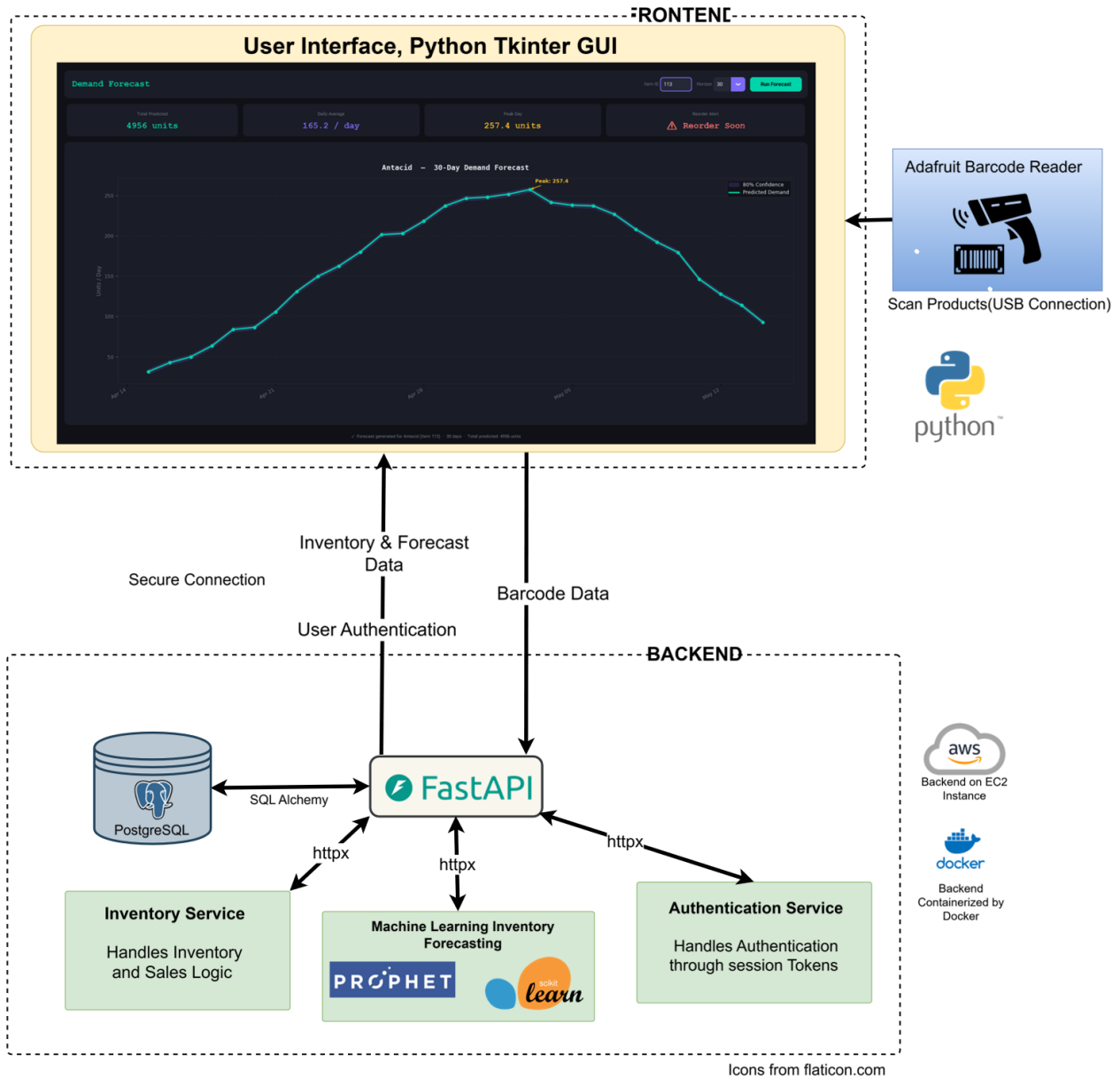


Figure 4-1 Architecture Diagram

4.1 Authentication Service

The Authentication Service is a FastAPI application responsible for managing user accounts and login sessions. It exposes three endpoints: POST /api/auth/login, POST /api/auth/logout, and GET /api/auth/me. It persists data to its own dedicated PostgreSQL database. The implementation details of this service are covered in depth in Section 7.

4.2 Inventory Service

The Inventory Service is a FastAPI application that manages the drug inventory and stock movement ledger. It provides full CRUD operations on inventory items and records every stock change as an immutable movement record. It also supports barcode-based item lookup. The implementation details are covered in depth in Section 7.

4.3 Forecasting Service

The Forecasting Service is a FastAPI application that exposes a single endpoint: GET /forecast/{item_id}. It fetches historical sales data from the Inventory Service, processes it into a daily time series, and passes it to Facebook Prophet to generate a demand forecast. The service is stateless and has no database of its own.

4.4 Frontend

The frontend is a desktop GUI application built with CustomTkinter. It communicates with the three backend services over HTTP and is structured as a tabbed interface covering Inventory, Forecast, Metrics, Tasks, and Account. All API calls are made on background threads to keep the UI responsive.

5 Project Plan

The project was managed using an Agile/Scrum methodology with weekly stand-ups and sprint planning sessions. Tasks were tracked in Jira, with each sprint lasting one to two weeks. The project was divided into five main phases:

- Phase 1 — Research and Requirements: Literature review, technology selection, system design, and initial project proposal.
- Phase 2 — Backend Development: Implementation of the Inventory Service and Authentication Service, including database schema design, API endpoints, and Dockerisation.
- Phase 3 — AI/ML Development: Implementation of the Forecasting Service, integration of Prophet and the Random Forest classifier, and testing with synthetic data.
- Phase 4 — Frontend Development: Implementation of the desktop GUI, integration with all three backend services, and barcode scanner integration.
- Phase 5 — Testing, CI/CD, and Deployment: Writing automated test suites, setting up GitHub Actions pipelines, and deploying to AWS EC2.

6 Implementation

This section covers the key technical decisions and implementation details of the SmartStock system, focusing on areas where specific design choices were made and justified

6.1 Database Design

SmartStock uses two independent PostgreSQL databases — one for the Inventory Service and one for the Authentication Service. Each service owns its data exclusively, meaning no service can access another's database directly.

6.1.1 Inventory Database

The Inventory Service uses a PostgreSQL database with two tables: `inventory` and `stock_movements`. The design follows a ledger pattern — the `inventory` table holds the current state of each item, while `stock_movements` acts as an immutable record of every change to that state.

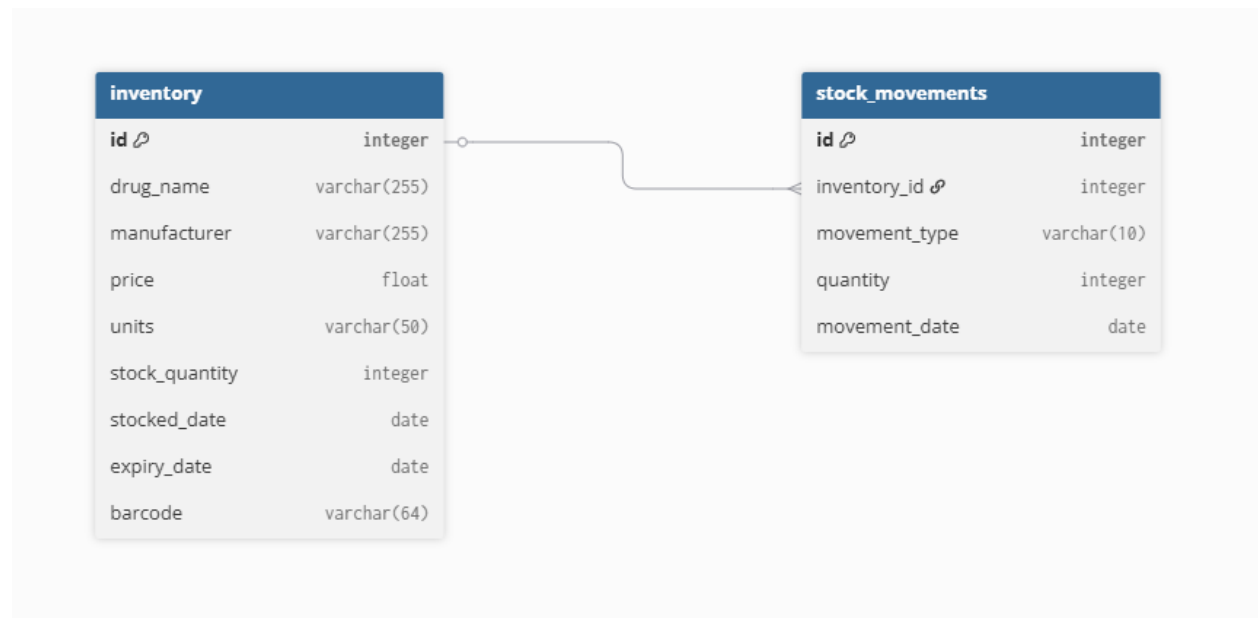


Figure 6.1 Inventory ER Diagram

The **inventory table** stores one row per drug item. Each row captures the drug name, manufacturer, units, price, current stock quantity, stocked date, expiry date, and a unique barcode. The barcode column has a unique constraint enforced at the database level, meaning the API will return a 409 conflict if an attempt is made to create two items with the same barcode. This prevents duplicate entries from being created regardless of how the API is called.

The `stock_quantity` column represents the current quantity on hand. It is updated every time a stock movement is recorded — incremented for IN movements and decremented for OUT movements. This means at any point in time, `stock_quantity` reflects the live stock level without needing to sum the movements table.

```

class InventoryDB(Base):
    __tablename__ = "inventory"

    id: Mapped[int] = mapped_column(primary_key=True)
    drug_name: Mapped[str] = mapped_column(String(255), nullable=False)
    manufacturer: Mapped[str] = mapped_column(String(255), nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    units: Mapped[str] = mapped_column(String(50), nullable=False)

    # Current stock on hand (state)
    stock_quantity: Mapped[int] = mapped_column(Integer, nullable=False)

    stocked_date: Mapped[date] = mapped_column(Date, nullable=False) # first received/created
    expiry_date: Mapped[date] = mapped_column(Date, nullable=False)

    barcode: Mapped[str] = mapped_column(String(64), unique=True, nullable=False)

    # History (ledger)
    movements: Mapped[List["StockMovementDB"]] = relationship(
        back_populates="inventory",
        cascade="all, delete-orphan"
    )

```

Figure 6.2 Inventory Database Model

The **stock_movements** table is the most important design decision in the Inventory Service. Rather than simply updating the stock quantity directly, every change to stock is recorded as a separate movement row with a type (IN or OUT), a quantity, and a date. IN movements represent restocks or deliveries. OUT movements represent sales or usage.

This approach was chosen for two reasons. First, it preserves a complete history of every stock change, which is essential for the demand forecasting component — Prophet requires historical sales data to generate a forecast, and without this ledger there would be no data to train on. Second, it provides auditability, which is particularly important in a pharmaceutical context where stock discrepancies may need to be investigated.

When a new inventory item is created, its initial stock quantity is automatically logged as an IN movement with the stocked date. This ensures the movements table is complete from day one.

```
class StockMovementDB(Base):
    __tablename__ = "stock_movements"

    id: Mapped[int] = mapped_column(primary_key=True)
    inventory_id: Mapped[int] = mapped_column(ForeignKey("inventory.id"), nullable=False)

    # IN = received/restock, OUT = sale/usage
    movement_type: Mapped[str] = mapped_column(String(10), nullable=False) # "IN" or "OUT"

    # store quantity as a positive number (cleaner)
    quantity: Mapped[int] = mapped_column(Integer, nullable=False)

    movement_date: Mapped[date] = mapped_column(Date, nullable=False)

    inventory: Mapped["InventoryDB"] = relationship(back_populates="movements")
```

Figure 6.3 Stock Movement Database Model

6.1.2 Authentication Database

The Authentication Service uses its own dedicated PostgreSQL database, completely separate from the Inventory database. This is a deliberate microservices design principle — each service owns its data and no other service can access it directly. If the two services shared a database, they would become tightly coupled and could not be deployed or scaled independently.

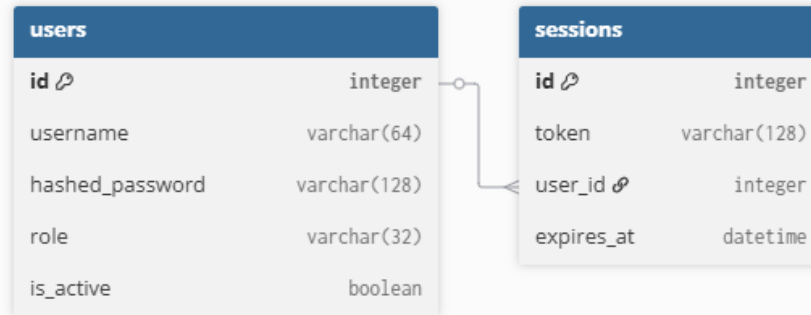


Figure 6.4 Authentication ER Diagram

The **users** table stores one row per user account. Each row holds a username, a hashed password, a role, and an active flag. The username column has a unique constraint so no two accounts can share the same username.

Passwords are never stored in plain text. Before a password is saved, it is hashed using SHA-256 via Python's *hashlib* module. When a user logs in, the submitted password is hashed using the same algorithm and compared against the stored hash. This means even if the database were compromised, the original passwords could not be recovered directly.

The *is_active* flag allows an account to be disabled without deleting it. This is important in a staff management context — if a staff member leaves, their account can be deactivated immediately without losing the audit trail of actions they performed while active.


```

class UserDB(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(64), unique=True, nullable=False)
    hashed_password: Mapped[str] = mapped_column(String(128), nullable=False)
    role: Mapped[str] = mapped_column(String(32), nullable=False, default="staff")
    is_active: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)

    sessions: Mapped[List["SessionDB"]] = relationship(
        back_populates="user",
        cascade="all, delete-orphan",
    )

```

Figure 6.5 UserDB Model

The **sessions** table stores one row per active login session. Each row holds a token, a reference to the user it belongs to via a foreign key, and an expiry timestamp. The token column has a unique constraint since no two active sessions can share the same token.

When a user logs in successfully, a new session row is created with a cryptographically random 64-character hex token generated using Python's `secrets.token_hex(32)`. This token is returned to the client, which stores it in memory and includes it in subsequent requests. The server validates incoming tokens by looking them up in the sessions table and checking that the expiry timestamp has not passed.

This approach was chosen over JWT because SmartStock is a pharmacy system where the ability to immediately revoke access is important. A JWT token is self-contained and remains valid until it expires — there is no way to invalidate it early without maintaining a blocklist, which partially defeats the purpose. A session token, by contrast, can be invalidated instantly by deleting its row from the database. This is what happens on logout.

The relationship between **users** and **sessions** is configured with cascade delete, meaning when a user is deleted, all their sessions are automatically deleted too.

6.1.3 Containerisation of Database

Both the Inventory and Authentication databases follow the same containerisation pattern. Each run as a dedicated PostgreSQL 17 container connected to its API service via a private Docker network, meaning the database port is never exposed to the outside world — only its corresponding API container can reach it. A named Docker volume is used to persist the database files on disk, ensuring data survives container restarts and image updates. The API container is configured to wait for the database to pass a health check before starting, preventing connection errors during startup.

6.2 Demand Forecasting

The forecasting pipeline is implemented in the Forecasting Service and consists of three stages: data retrieval, data preparation, and model fitting and prediction.

6.2.1 Data Retrieval

When a forecast is requested, the Forecasting Service makes a GET request to the Inventory Service's `/api/movements` endpoint, filtering by `item_id` and `movement_type=OUT` to retrieve only the sales history for the requested item. This was an important optimisation — the original implementation fetched all movements and filtered in Python, which became slow as the movements table grew. Moving the filter to the database query reduced response times significantly.

The service requires a minimum of two data points to proceed. If no sales history exists or only one record is found, the endpoint returns a descriptive error rather than attempting to fit a model on insufficient data.

```
def fetch_movements(item_id: int) -> list[dict]:
    """
    Fetches all OUT (sale) movements for a given inventory item
    from the inventory microservice.

    Returns a list of dicts with 'movement_date' and 'quantity'.
    """
    try:
        response = requests.get(
            f"{INVENTORY_API}/api/movements",
            params={"item_id": item_id, "limit": 1000},
            timeout=30
        )
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        raise RuntimeError(f"Failed to reach inventory service: {e}")

    all_movements = response.json()

    # Just return all results directly – already filtered by the API
    sales = [
        {
            "movement_date": m["movement_date"],
            "quantity": m["quantity"]
        }
        for m in response.json()
        if m["movement_type"] == "OUT"
    ]

    return sales
```

Figure 6.6 Fetch Movements

6.2.2 Data Preparation

Before Prophet can be used, the raw movement data must be transformed into a clean daily time series. The following steps are applied:

- The movement dates and quantities are loaded into a pandas Data Frame.
- Multiple sales on the same day are aggregated by summing their quantities.
- Missing days between the first and last sale are filled with zero demand, days with no recorded sales are treated as zero demand, not missing data
- The Data Frame is renamed to ds and y columns as required by Prophet

This preparation step is critical. Prophet expects a complete, evenly spaced time series. Without filling missing days, the model would produce unreliable forecasts.

```
# --- 1. Build a daily aggregated DataFrame ---
df = pd.DataFrame(movements)
df["ds"] = pd.to_datetime(df["movement_date"])
df["y"] = df["quantity"]

# Aggregate by day (multiple sales on same day get summed)
df = df.groupby("ds", as_index=False)["y"].sum()

# Fill in missing days with 0 (no sales = 0 demand that day)
full_range = pd.date_range(df["ds"].min(), df["ds"].max(), freq="D")
df = df.set_index("ds").reindex(full_range, fill_value=0).reset_index()
df.columns = ["ds", "y"]
```

Figure 6.7 Data Preparation

6.2.3 Model Fitting and Prediction

Prophet is configured with the following settings:

- Weekly seasonality enabled — pharmacy sales often follow a weekly pattern
- Yearly seasonality enabled — accounts for seasonal demand changes
- Daily seasonality disabled — not meaningful for inventory data
- 80% confidence interval — provides upper and lower bounds on the forecast

Prophet is fitted to the prepared time series and used to predict daily demand for the requested horizon (default 30 days). Only the future portion of the forecast is returned — Prophet also predicts over the historical period as part of its fitting process, but this is filtered out before the response is sent.

All predicted values are clamped to a minimum of zero since demand cannot be negative. The final response includes the predicted date, predicted quantity (yhat), and the lower and upper

confidence bounds (yhat_lower, yhat_upper).

```
# --- 2. Fit Prophet ---
model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=True,
    daily_seasonality=False, # daily rarely useful for inventory
    interval_width=0.80      # 80% confidence interval
)
model.fit(df)

# --- 3. Generate future dataframe and predict ---
future = model.make_future_dataframe(periods=horizon, freq="D")
forecast = model.predict(future)
```

Figure 6.8 Model Fitting and Prediction

6.2.4 Forecast Visualisation

The forecast results are displayed in the GUI's Forecast tab as an interactive matplotlib chart embedded directly in the CustomTkinter window. The chart shows:

- The predicted demand line
- The 80% confidence band shaded around the line
- A peak demand annotation marking the highest predicted day
- Summary cards showing total predicted units, daily average, peak day value, and a reorder alert



Figure 6.7 Forecast Visualization

6.3 Stockout Risk Prediction – Research

As part of the research phase of this project, a Random Forest classifier was trained to predict which pharmacy inventory items are at risk of stocking out within the next 30 days. The model was trained on live data pulled directly from the SmartStock Inventory Service API.

Stockout prediction was framed as a binary classification problem. Given the current state of an inventory item, the model predicts one of two outcomes:

- At Risk — the item is likely to stock out within 30 days
- Safe — the item has sufficient stock for the next 30 days

The label was derived deterministically: if the current stock quantity divided by the average daily usage rate was 30 days or fewer, the item was labelled At Risk. This threshold was chosen to give pharmacy staff sufficient lead time to reorder.

6.3.1 Feature Engineering

Four features were extracted from the inventory and stock movements data:

- `stock_quantity` — current units on hand
- `avg_daily_usage` — mean units sold per day over the recorded history
- `days_since_restock` — days elapsed since the last IN movement
- `days_until_expiry` — days remaining before the item expires

The feature distribution charts confirm that the two classes are meaningfully separable — At Risk items cluster towards lower stock quantities and higher daily usage rates, which aligns with intuition. The correlation matrix shows that `stock_quantity` has a moderate negative correlation with the `at_risk` label (-0.49), while `avg_daily_usage` has a moderate positive correlation (0.54), confirming both are genuinely informative features.

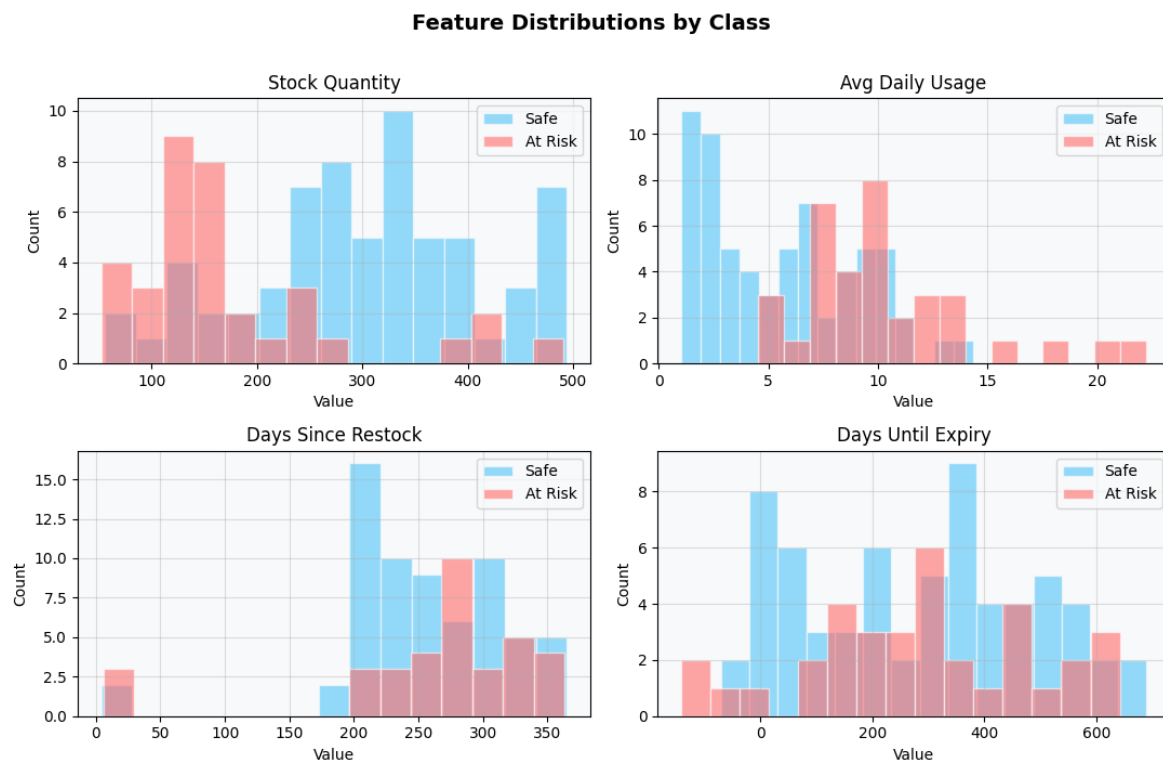


Figure 6.8 Feature Distribution by Class

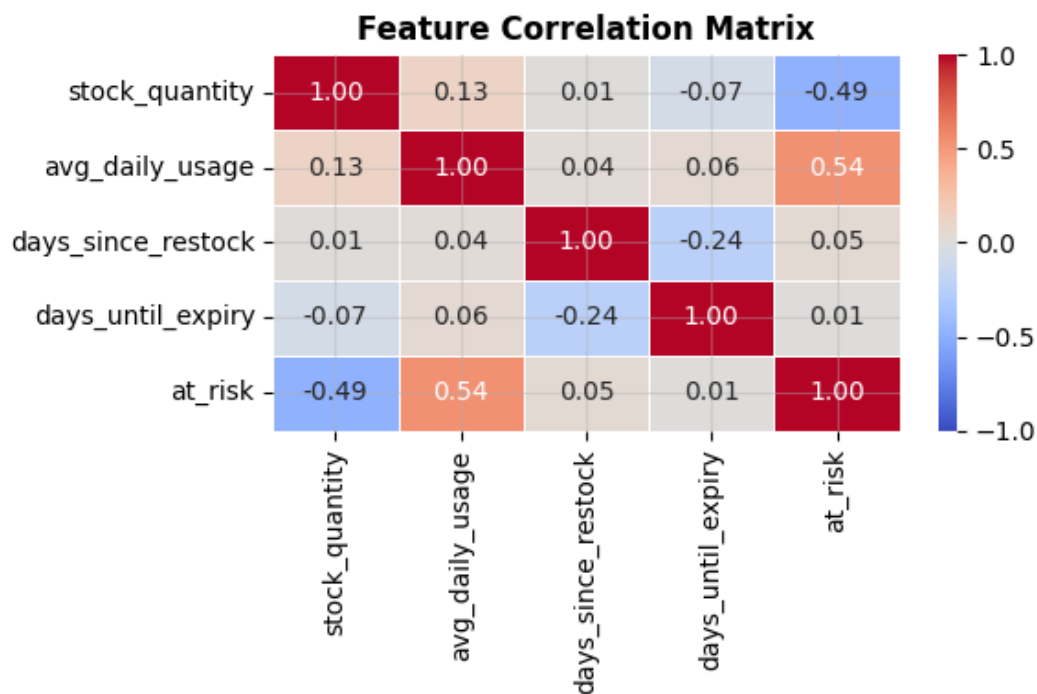


Figure 6.9 Feature Correlation Matrix

6.3.2 Results

The model was trained on 80 items and evaluated on a held-out test set of 20 items, achieving 90% test accuracy. The confusion matrix shows 13 Safe items correctly identified, 5 At Risk items correctly identified, and 2 false negatives — At Risk items incorrectly classified as Safe. In a pharmacy context, false negatives are the costliest error type, as they represent items that will unexpectedly run out. The classification report shows the model achieved a precision of 1.00 and recall of 0.71 for the At-Risk class, meaning every item it flagged as At Risk was genuinely at risk, but it missed 2 out of 7 genuinely at-risk items. The ROC AUC of 0.97 indicates strong overall discriminative ability.

5-fold cross-validation yielded a mean accuracy of 91% ($\pm 9.7\%$), with individual fold scores ranging from 75% to 100%, reflecting the inherent variability of a 100-item dataset.

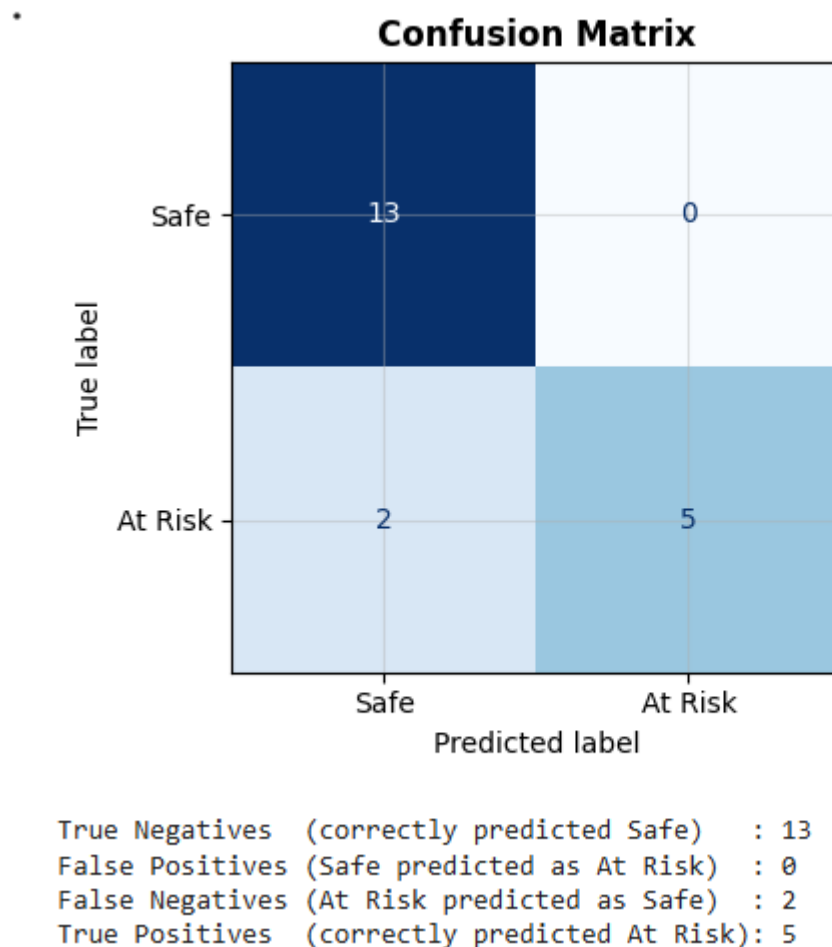


Figure 6.10 Confusion Matrix

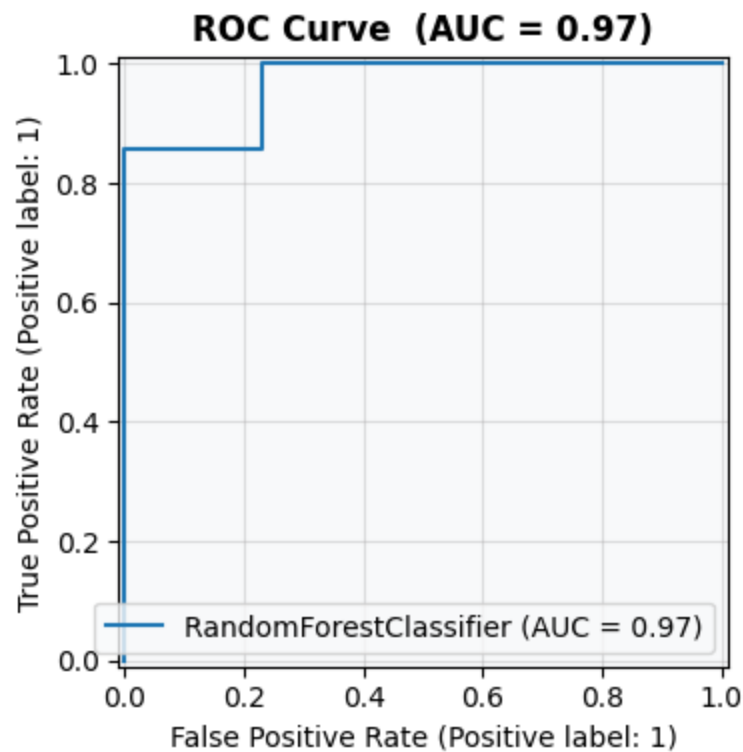


Figure 6.11 ROC Curve

	precision	recall	f1-score	support
Safe	0.87	1.00	0.93	13
At Risk	1.00	0.71	0.83	7
accuracy			0.90	20
macro avg	0.93	0.86	0.88	20
weighted avg	0.91	0.90	0.90	20

Figure 6.12 Classification report

5-fold cross-validation accuracy:

Fold 1: 75.00%
Fold 2: 100.00%
Fold 3: 95.00%
Fold 4: 100.00%
Fold 5: 85.00%

Mean : 91.00%

Std : 9.70%

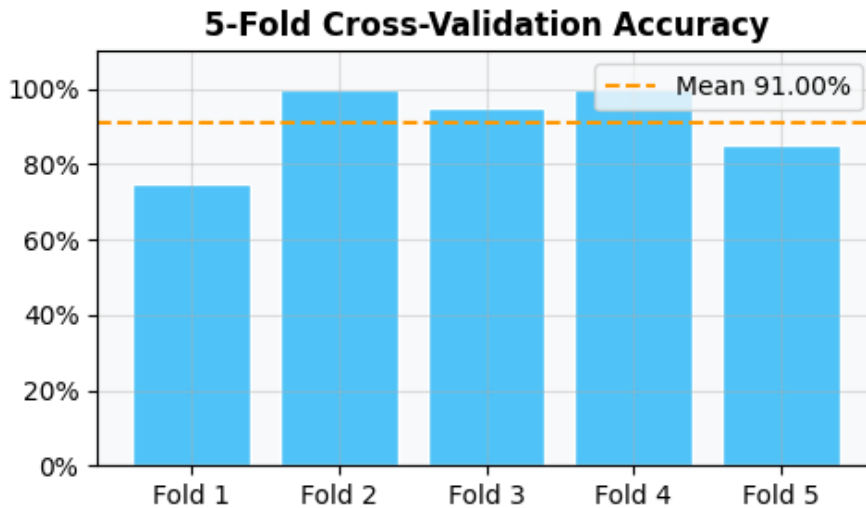


Figure 6.13 Feature Distribution by Class

Feature Importances

The feature importance scores confirm that *avg_daily_usage* (43.8%) and *stock_quantity* (39.4%) are the dominant predictors, together accounting for over 83% of the model's decision-making. *days_since_restock* (9.8%) and *days_until_expiry* (7.0%) contribute meaningfully once the seed data included varied restock histories, validating the importance of realistic training data.

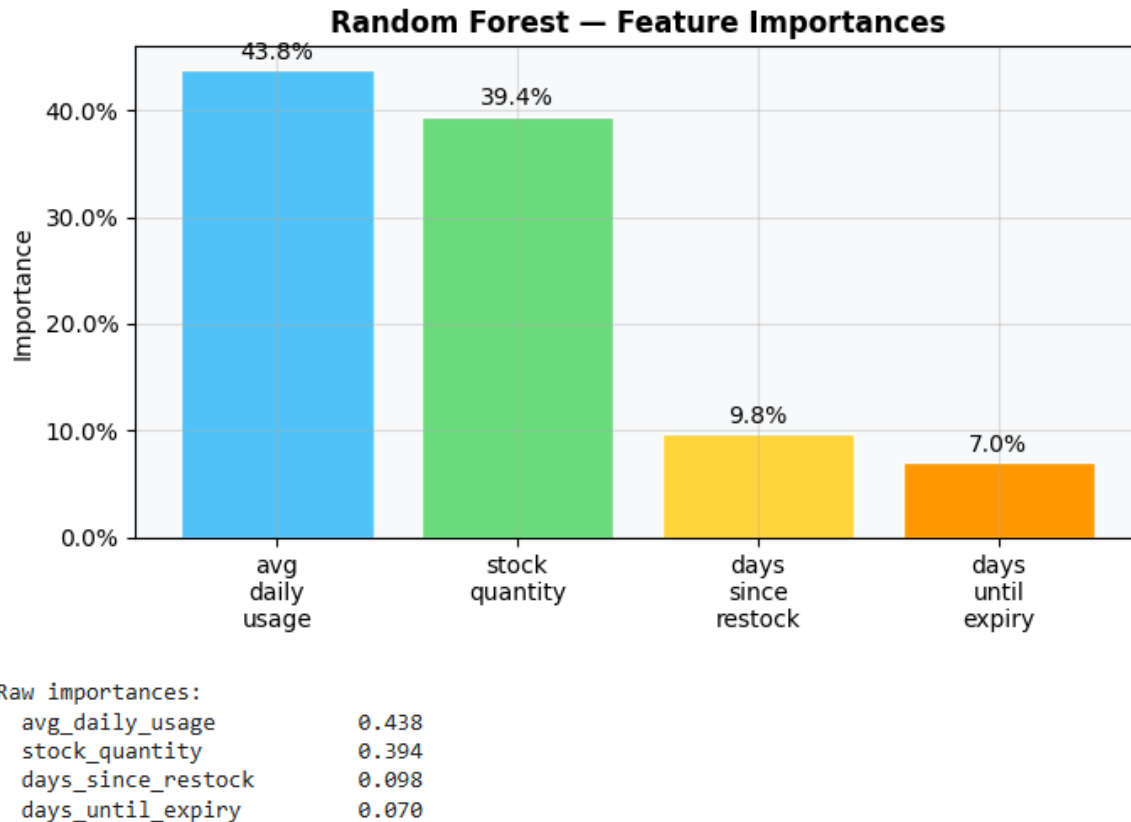


Figure 6.14 Classification report

Relationship to Prophet

The two models answer fundamentally different questions and are designed to be used together. Prophet provides a detailed view of future demand for a specific item over time. The Random Forest provides a quick at-a-glance risk flag across all items simultaneously, as shown in the predicted 30-day stockout risk chart. A staff member could use the classifier output to identify which items need attention, then use Prophet to investigate the demand trajectory of those specific items in detail.

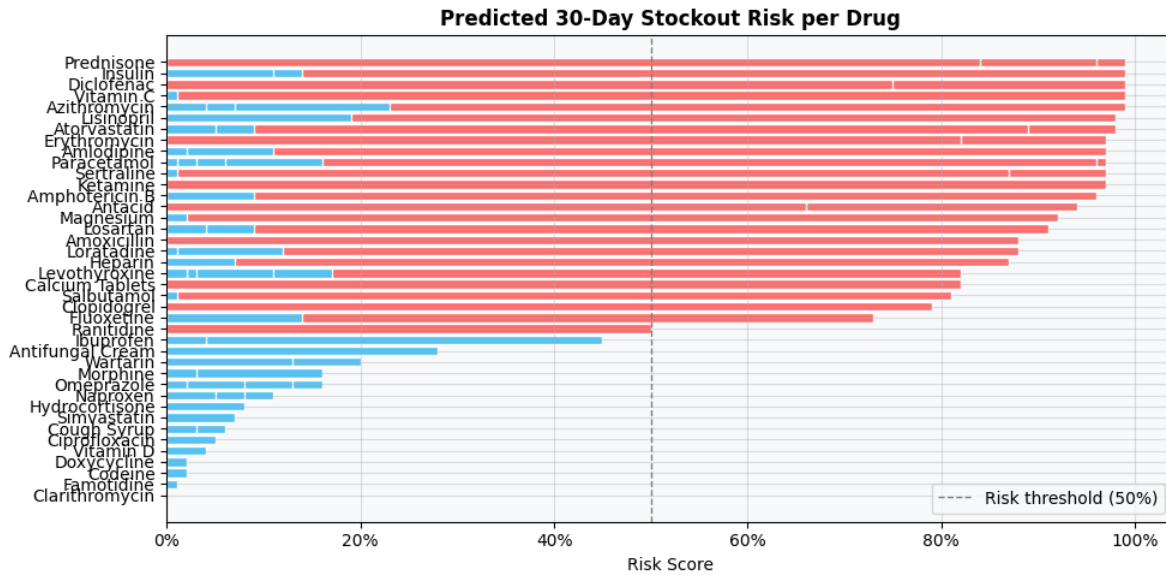


Figure 6.15 Adafruit Barcode Reader

Current Status

The classifier was successfully trained and evaluated on live data from the SmartStock system, producing strong results.

6.4 Barcode Integration

SmartStock integrates an Adafruit MCR12 USB barcode scanner to allow pharmacy staff to look up inventory items instantly without typing. When a barcode is scanned, the system retrieves and displays the matching drug's details in the GUI automatically.

6.4.1 Hardware



Figure 6.16 Adafruit Barcode Reader []

The Adafruit MCR12 is a USB barcode scanner that supports two operating modes: HID Keyboard mode, where it types the scanned barcode into whatever window is active, and USB Virtual COM Port mode, where it appears as a serial port that software can read from programmatically [REF]. For SmartStock, the scanner was configured to use Virtual COM Port mode using configuration barcodes from the manufacturer's manual [REF]. This allows the Python backend to listen on the serial port and react to scans in real time rather than relying on keyboard input.

The scanner was set to a baud rate of 9600 (the device default) and configured to support EAN-13 barcodes [REF], which is the 13-digit format used on most commercial pharmaceutical packaging.

6.4.2 Setup Challenges

During initial setup, the scanner was not visible in Windows Device Manager under any category, not under Ports, Keyboards, or HID devices. After testing different USB ports and confirming the cable supported data transfer rather than power only, the issue was resolved by switching the scanner from its default HID Keyboard mode to Virtual COM Port mode using the configuration barcodes in the manual [REF] and re-plugging the device.

A dedicated endpoint was added to the Inventory Service to handle barcode lookups:

POST /api/inventory/scan

The endpoint accepts a JSON body containing the barcode string, queries the inventory table for a matching record, and returns the full item details. The barcode column has a unique constraint at the database level, ensuring lookups are always unambiguous.

A route ordering bug was identified and fixed during development — in FastAPI, the /api/inventory/scan route must be defined before the /{item_id} route, otherwise the string "scan" is interpreted as an item ID and the request returns a 422-validation error [REF].

```
@app.post("/api/inventory/scan", response_model=InventoryRead)
async def scan_barcode(request: Request, db: Session = Depends(get_db)):
    """
    Look up an inventory item by barcode.
    Used by the barcode scanner to identify items quickly.
    Returns 400 if no barcode is provided, 404 if not found.
    """
    payload = await request.json()
    barcode = payload.get("barcode")
    if not barcode:
        raise HTTPException(status_code=400, detail="Barcode not provided")

    item = db.query(InventoryDB).filter(InventoryDB.barcode == barcode).first()
    if not item:
        raise HTTPException(status_code=404, detail="Item not found")
    return item
```

Figure 6.17 Adafruit Barcode Reader

6.4.3 Frontend

A dedicated Barcode Service class runs a background thread in the GUI that continuously listens on the serial port using the PySerial library [REF]. When a barcode arrives, it fires a callback to any registered listener. The Inventory Tab registers a callback that highlights the matching item in the list and displays its details, giving the user immediate visual feedback. If

the scanned barcode does not match any item in the database, a modal is shown allowing the user to assign it to an existing item.

[Screenshot: Inventory tab showing a barcode scan]

6.5 GUI Implementation

The SmartStock frontend is a desktop application built with CustomTkinter [REF], a modern themed wrapper around Python's built-in Tkinter library [REF]. It communicates with the three backend services over HTTP using the Requests library [REF] and is structured as a tabbed interface.

6.5.1 Application Structure

The application entry point is `app.py`, which displays a login screen on startup. On successful authentication, the login screen is replaced by the main tabbed interface. The tabs are managed by a `MainTabView` class that loads each tab as a separate module:

- Inventory — browse, search, and manage stock items; integrates barcode scanning
- Forecast — select an item and generate a Prophet [REF] demand forecast with an embedded matplotlib [REF] chart
- Metrics — stock overview and key statistics
- Tasks — task management
- Account — user account details

6.5.2 Threaded API Calls

All API calls are made on background threads rather than the main UI thread. This is a deliberate design decision, if network requests were made on the main thread, the GUI would freeze while waiting for the server to respond, making the application feel unresponsive. Python's threading module [REF] is used to run each request in the background, with the result passed back to the main thread via a callback function once the response arrives.

6.5.3 Barcode Integration

A dedicated Barcode Service class runs a persistent background thread that listens on the serial port for incoming scans using the PySerial library [REF]. It uses a register/unregister callback pattern so that any tab can subscribe to scan events. When the Inventory Tab is active and a barcode is scanned, the matching item is highlighted in the list automatically.

6.5.4 Login Screen

The login screen collects a username and password and sends them to the Authentication Service. HTTP error codes are mapped to user-friendly messages — a 401 returns "Invalid

credentials" and 403 returns "Account disabled". A loading state is shown while the request is in flight to prevent double submission.

[Screenshot: Login screen]

[Screenshot: Main tabbed interface]

[Screenshot: Forecast tab with embedded chart]

6.6 Deployment

SmartStock is deployed on an AWS EC2 instance, making all three backend services accessible over the internet. This allows the desktop frontend to connect to live backend services without needing to run them locally.

6.6.1 Infrastructure

A *t3.micro* EC2 instance was provisioned on AWS running Ubuntu. A permanent Elastic IP address was assigned to the instance so that the public IP address does not change between reboots. Inbound firewall rules were configured to allow traffic on ports 8001 (Authentication Service), 8002 (Inventory Service), and 8003 (Forecasting Service) from any IP address, and port 22 (SSH) restricted to the developer's IP for secure remote access.

6.6.2 Containerisation

All three backend services are containerised using Docker and published to the GitHub Container Registry (ghcr.io) as part of the CI/CD pipeline. A dedicated deployment repository (SmartStock_Deployment) contains a single docker-compose.yml file that pulls all three service images and starts them together on the EC2 instance with a single command:

```
docker compose up -d
```

Each service runs in its own container on a private Docker network, meaning the database ports are never exposed to the outside world, only the API ports are accessible externally. Named Docker volumes ensure that database data persists across container restarts and image updates.

```

services:
# — Authentication Service —————
auth-db:
  image: postgres:17-alpine
  container_name: smartstock_auth_db
  environment:
    POSTGRES_USER: app
    POSTGRES_PASSWORD: app
    POSTGRES_DB: authdb
  volumes:
    - auth-postgres-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U app -d authdb"]
    interval: 5s
    retries: 10
  restart: unless-stopped
  networks:
    - smartstock-net

auth-service:
  image: ghcr.io/anesuleo/smartstock_authentication/auth:latest
  container_name: smartstock_auth
  environment:
    APP_ENV: docker
    DATABASE_URL: postgresql+psycopg2://app:app@auth-db:5432/authdb
  depends_on:
    auth-db:
      condition: service_healthy
  ports:
    - "8001:8000"
  restart: unless-stopped
  networks:
    - smartstock-net

# — Inventory Service —————
inventory-db:
  image: postgres:17-alpine
  container_name: smartstock_inventory_db
  environment:
    POSTGRES_USER: app
    POSTGRES_PASSWORD: app
    POSTGRES_DB: appdb

```

Figure 6.18 docker-compose.yml from Deployment

6.6.3 Updating the Server

When a new version of a service is deployed, the process on the EC2 instance is:

docker compose pull

docker compose up -d

Docker pulls the latest image for each service and restarts any containers whose image has changed, leaving the others untouched.

6.7 CI/CD Pipeline

Each SmartStock service repository has a GitHub Actions [REF] CI/CD pipeline that runs automatically on every push to the master branch. The pipeline ensures that no broken code is ever deployed to the server, a new Docker image is only built and published if all tests pass first.

The pipeline consists of two jobs that run in sequence:

6.7.1 Test Job

The first job runs on an Ubuntu runner and executes the following steps:

- Checks out the repository code
- Sets up Python 3.11
- Installs all dependencies from requirements.txt
- Runs the full pytest [REF] test suite with APP_ENV=test and an in-memory SQLite database [REF], so no real database instance is needed in CI

If any test fails, the pipeline stops here and the Docker image is not built.

6.7.2 Build and Push Job

The second job only runs if the test job passes, and only on direct pushes to master — not on pull requests. It:

- Logs in to the GitHub Container Registry (ghcr.io) [REF] using the built-in GITHUB_TOKEN, so no manual credentials need to be stored
- Builds the Docker image from the repository's Dockerfile
- Pushes the image to ghcr.io/anesuleo/smartstock_inventory/inventory:latest

Once the image is published, the EC2 server can be updated by running `docker compose pull` followed by `docker compose up -d`, which pulls the new image and restarts the affected

container.

The figure displays two screenshots of the GitHub Actions interface, showing the execution details of two different workflows.

Top Screenshot: 'test' workflow

- Workflow:** test
- Status:** succeeded yesterday in 16s
- Annotations:** 1 warning
- Steps:**
 - Set up job (1s)
 - Run actions/checkout@v4 (1s)
 - Set up Python (0s)
 - Install dependencies (10s)
 - Run tests (1s)
 - Post Set up Python (0s)
 - Post Run actions/checkout@v4 (1s)
 - Complete job (0s)

Bottom Screenshot: 'build-and-push' workflow

- Workflow:** build-and-push
- Status:** succeeded yesterday in 33s
- Annotations:** 1 warning
- Steps:**
 - Set up job (2s)
 - Run actions/checkout@v4 (0s)
 - Log in to GitHub Container Registry (1s)
 - Build and push Docker image (28s)
 - Post Build and push Docker image (0s)
 - Post Log in to GitHub Container Registry (1s)
 - Post Run actions/checkout@v4 (0s)
 - Complete job (0s)

Figure 6.19 GitHub Actions workflow

7 Testing

Testing was a core part of the SmartStock development process. Both the Authentication Service and the Inventory Service have dedicated pytest test suites that run automatically on every push to the master branch as part of the CI/CD pipeline.

7.1 Test Setup

Both test suites use FastAPI's built-in TestClient [REF] combined with an in-memory SQLite database, meaning no real server or PostgreSQL instance is needed to run the tests. The real database dependency is replaced using FastAPI's dependency override mechanism, the `get_db` function is swapped out for a test version that connects to the in-memory database instead.

Each test starts with a completely clean database, created fresh before the test and dropped afterwards, ensuring tests are fully isolated from one another.

7.2 Inventory Service Tests

The Inventory Service has 19 tests covering all major API endpoints and business logic:

Category	Tests
Health check	API returns ok status
Create Inventory	Successful creation, duplicate barcode rejected with 409
List Inventory	Empty list, item appears after creation
Get single item	Correct item returned by ID 404 for unknown ID
Update Inventory	Full replacement with PUT, partial update with PATCH
Delete Inventory	Item deleted and no longer retrievable, 404 for unknown ID
Barcode scan	Valid barcode returns item, unknown barcode returns 404 for unknown ID
Stock movements	IN movement increases stock, OUT movement decreases stock, OUT exceeding stock returns 400
Sales and movements	Sales endpoint returns only OUT movements, movements endpoint returns all movements

```

✓ Run tests

1  ▶ Run pytest tests/ -v
13 ===== test session starts =====
14 platform linux -- Python 3.11.15, pytest-8.4.2, pluggy-1.6.0 -- /opt/hostedtoolcache/Python/3.11.15/x64/bin/python
15 cachedir: .pytest_cache
16 rootdir: /home/runner/work/SmartStock_Inventory/SmartStock_Inventory
17 configfile: pytest.ini
18 plugins: anyio-4.13.0
19 collecting ... collected 19 items
20
21 tests/test_inventory.py::test_health PASSED [ 5%]
22 tests/test_inventory.py::test_create_inventory PASSED [ 10%]
23 tests/test_inventory.py::test_create_inventory_duplicate_barcode PASSED [ 15%]
24 tests/test_inventory.py::test_list_inventory_empty PASSED [ 21%]
25 tests/test_inventory.py::test_list_inventory PASSED [ 26%]
26 tests/test_inventory.py::test_get_inventory_item PASSED [ 31%]
27 tests/test_inventory.py::test_get_inventory_item_not_found PASSED [ 36%]
28 tests/test_inventory.py::test_update_inventory PASSED [ 42%]
29 tests/test_inventory.py::test_patch_inventory PASSED [ 47%]
30 tests/test_inventory.py::test_delete_inventory PASSED [ 52%]
31 tests/test_inventory.py::test_delete_inventory_not_found PASSED [ 57%]
32 tests/test_inventory.py::test_scan_barcode PASSED [ 63%]
33 tests/test_inventory.py::test_scan_barcode_not_found PASSED [ 68%]
34 tests/test_inventory.py::test_scan_barcode_missing PASSED [ 73%]
35 tests/test_inventory.py::test_add_stock_in PASSED [ 78%]
36 tests/test_inventory.py::test_add_stock_out PASSED [ 84%]
37 tests/test_inventory.py::test_add_stock_out_insufficient PASSED [ 89%]
38 tests/test_inventory.py::test_list_sales PASSED [ 94%]
39 tests/test_inventory.py::test_list_movements PASSED [100%]
40
41 ===== 19 passed in 0.71s =====

```

Figure 6.20 Inventory Tests Passing

7.3 Authentication Service Tests

The Authentication Service has 9 tests covering the login, logout, and session validation endpoints:

Category	Tests
Login	Successful login returns token, invalid credentials return 401, disabled account returns 403
Session validation	Valid token returns user details, invalid token returns 401, expired token returns 401
Logout	Logout invalidates token, subsequent requests with same token return 401, unknown token handled silently

```

✓ Run tests

1  ▶ Run pytest tests/ -v
13  ===== test session starts =====
14  platform linux -- Python 3.11.15, pytest-8.4.2, pluggy-1.6.0 -- /opt/hostedtoolcache/Python/3.11.15/x64/bin/python
15  cachedir: .pytest_cache
16  rootdir: /home/runner/work/SmartStock_Authentication/SmartStock_Authentication
17  configfile: pytest.ini
18  plugins: anyio-4.13.0
19  collecting ... collected 8 items
20
21  tests/test_auth.py::test_login_success PASSED [ 12%]
22  tests/test_auth.py::test_login_wrong_password PASSED [ 25%]
23  tests/test_auth.py::test_login_unknown_user PASSED [ 37%]
24  tests/test_auth.py::test_login_disabled_account PASSED [ 50%]
25  tests/test_auth.py::test_me_valid_token PASSED [ 62%]
26  tests/test_auth.py::test_me_invalid_token PASSED [ 75%]
27  tests/test_auth.py::test_me_expired_token PASSED [ 87%]
28  tests/test_auth.py::test_logout_invalidates_token PASSED [100%]
29
30  ===== 8 passed in 0.54s =====

```

Figure 6.21 Authentication Tests Passing

7.4 Running the tests

Tests can be run locally with:

```
pytest tests/ -v
```

In CI, the APP_ENV=test environment variable is set so the application loads the test database configuration automatically. All 28 tests across both services must pass before a new Docker image is built and deployed.

8 Challenges Faced

Several technical challenges were encountered during the development of SmartStock. This section documents the most significant ones and how they were resolved.

8.1 Barcode Scanner Detection

During the initial setup of the Adafruit MCR12 barcode scanner, the device was not visible in Windows Device Manager under any category, not under Ports, Keyboards, or HID devices. After testing different USB cables and ports, the issue was traced to the scanner being in HID Keyboard mode by default. Switching the device to USB Virtual COM Port mode using configuration barcodes from the manufacturer's manual [REF] resolved the detection issue and allowed the Python serial listener to communicate with the scanner reliably.

8.2 FastAPI Route Ordering Bug

A subtle bug was discovered in the Inventory Service where the `/api/inventory/scan` endpoint was returning 422 Unprocessable Entity errors instead of the expected response. The root cause was that FastAPI matches routes in the order they are defined — because the `/item_id` route was defined before `/scan`, FastAPI was interpreting the string "scan" as an integer item ID and failing validation. Moving the `/api/inventory/scan` route definition above the `/item_id` route resolved the issue.

8.3 NFC Hardware Integration

The project originally planned to use an X-NUCLEO-NFC12A1 board connected to an Arduino Uno for NFC-based item scanning. The board's pin layout is designed for STM32 Nucleo boards, not Arduino Uno, meaning the boards could not simply be stacked — individual jumper wires were required to connect the correct pins. After establishing the wiring, register reads from the ST25R3911B chip were consistently returning 0x0, indicating a likely failed or poorly seated MISO line. Due to time constraints, NFC integration was descoped and replaced with the token authentication approach, which proved more reliable for the pharmacy use case.

8.4 Docker Memory Constraints on EC2

The Forecasting Service uses Facebook Prophet [REF] for time-series forecasting, which requires compiling C++ extensions during installation and consumes significant memory at runtime. On the `t3.micro` EC2 instance (1GB RAM), Prophet's memory usage caused instability during initial deployment. This was managed by ensuring no other memory-intensive operations ran concurrently during forecasting requests.

8.5 Synthetic Training Data Quality

During the research phase for the Random Forest stockout classifier, the initial seed script generated all inventory items with the same stocked date, resulting in zero variance in the `days_since_restock` feature. This meant the feature carried no discriminative signal for the model. The seed script was updated to randomise stocked dates across a one-year window and introduce periodic IN (restock) movements throughout each item's history, producing a more realistic dataset and allowing all four features to contribute meaningfully to the classifier.

8.6 Session Token vs JWT

An early design decision for the Authentication Service was whether to use JWT tokens or session tokens. JWT tokens are stateless and do not require a database lookup on every

request, but they cannot be invalidated before they expire, a significant limitation in a pharmacy context where staff accounts may need to be revoked immediately. Session tokens stored in the database were chosen instead, as they can be deleted instantly on logout or account deactivation, providing the immediate revocation capability required.

9 A.I Usage

AI tools were used throughout the development of this project in accordance with the module guidelines. All AI-generated content was reviewed, understood, and integrated into the project by the author. The following describes how AI was used and where.

9.1 Tools Used

- **Claude (Anthropic)** — used as the primary AI assistant throughout the project for code generation, debugging, report writing, and design decisions.
- **ChatGPT (OpenAI)** — used during the initial barcode scanner setup to document the troubleshooting process.

9.2 How AI Was Used

9.2.1 Code Generation

Claude was used to generate boilerplate code for the FastAPI endpoints, SQLAlchemy models, pytest test suites, and the GitHub Actions CI/CD pipeline. In all cases the generated code was reviewed, tested, and adapted to fit the specific requirements of SmartStock. No code was committed without being understood by the author.

9.2.2 Debugging

Claude assisted in diagnosing several bugs encountered during development, most notably the FastAPI route ordering issue that caused 422 errors on the `/api/inventory/scan` endpoint, and the NFC hardware MISO line diagnosis.

9.2.3 Report Writing

Claude was used to help structure and draft sections of this report. The author provided the technical content, decisions, and results — Claude assisted in articulating them clearly in written form.

9.2.4 Design Decisions

Claude was used as a sounding board for architectural decisions, such as the choice between JWT and session tokens for authentication, the decision to use Prophet for forecasting and Random Forest for stockout prediction, and the microservices architecture.

9.2.5 Acknowledgement

As required by the module guidelines, all AI usage is acknowledged here. The author is able to explain and demonstrate all parts of the project. GitHub commit history reflects steady, incremental progress throughout the year rather than sudden large additions, consistent with genuine engagement with the codebase.

10 Conclusion

This project set out to design and build an AI-powered inventory and demand forecasting system for use in pharmaceutical and warehouse stock management environments. The result is SmartStock — a fully functional, deployable system comprising four independently deployable microservices, a desktop GUI frontend, and two AI components.

The core objectives of the project were met. The Inventory Service provides a complete stock management backend with a ledger-based movement history that feeds directly into the forecasting pipeline. The Authentication Service provides session-based access control with immediate token revocation, appropriate for a pharmacy environment where staff accounts may need to be disabled instantly. The Forecasting Service uses Facebook Prophet to generate demand forecasts from historical sales data, presented to the user as an interactive chart with confidence intervals and a reorder alert. The desktop GUI brings all of these together in a clean tabbed interface with real-time barcode scanning support.

Beyond the core implementation, the project included a research component exploring the use of a scikit-learn Random Forest classifier for stockout risk prediction. Trained on live data from the SmartStock system, the classifier achieved 90% test accuracy and an AUC of 0.97, demonstrating that the stock movement ledger collected by the Inventory Service is sufficient to support meaningful machine learning on top of the existing infrastructure.

The project also demonstrated good software engineering practice throughout — containerised services, automated testing with pytest, a CI/CD pipeline using GitHub Actions, and deployment to AWS EC2. All 28 tests across the Authentication and Inventory services pass automatically on every push to the master branch, ensuring the system remains stable as it evolves.

Several challenges were encountered and overcome during development, including hardware integration issues with the barcode scanner, a subtle FastAPI route ordering bug, and limitations with training data quality for the machine learning component. Each of these is documented in Section 9.

Looking ahead, the most natural next steps for SmartStock would be to integrate the Random Forest classifier into the GUI as a risk dashboard, complete the NFC hardware integration that was descope due to time constraints, and expand the test coverage to include the Forecasting Service. With the microservices architecture and CI/CD pipeline already in place, these additions could be made incrementally without disrupting the existing system.

11 Appendix

12 References

[1] H. Kinsley, "Reinforcement Learning," PythonProgramming, [Online]. Available: <https://pythonprogramming.net/q-learning-reinforcement-learning-python-tutorial/>. [Accessed 02 02 2021].

[2] [Online].

[3] MakeSigns, "Scientific Posters Tutorial," [Online]. Available: <https://www.makesigns.com/tutorials/scientific-poster-parts.aspx>. [Accessed 09 02 2021].

[4] Arduino. [Online]. Available: <https://www.arduino.cc/>. [Accessed 09 02 2021].

[x] J Adafruit MCR12 product page — first mention of the hardware

MCR12 Scanner Manual (the PDF you linked) — for HID vs COM mode, baud rate, EAN-13 configuration, and the configuration barcodes

FastAPI routing documentation — for the route ordering issue

PySerial documentation — for the serial communication library [x] S. Lippman, J. Lajoie and B.E. Moo. "Classes" in *C++ Primer*. 5th ed. Massachusetts: Addison Wesley, 2013.

[x] IEEE Signal Processing Society. "Signal Processing for 5G", *YouTube*, 2019. [Online].

Available:

https://www.youtube.com/watch?v=uca6X4Ykcmg&list=PLcZOnmyqlalacL9YqkhyufLQGIW_C78Os&index=6. Accessed: Feb 2, 2021.

[x] Digilent. "Basys 3 Reference Manual", *Digilent Reference*. [Online]. Available:

<https://reference.digilentinc.com/basys3/refmanual>. Accessed: Feb 2, 2021.

[x] P. J. Ashenden. *Digital Design (Verilog): An Embedded Systems Approach Using Verilog*. Burlington: Morgan Kaufmann, 2007.

[x] M. Lynch. "Discrete Fourier Transform (DFT)", Lecture, Digital Signal Processing, Galway-Mayo Institute of Technology, Galway, 2020.

[x] MRC Centre for Global infectious Disease Analysis. (2021). *Covid-Sim*. [Source Code]. Available: <https://github.com/mrc-ide/covid-sim>. Accessed: Feb 2, 2021.

[x] OpenCV. "Face Detection Using Haar Cascades", *OpenCV-Python Tutorials*. [Online]. Available: https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_objdetect/py_face_detection/py_face_detection.html#face-detection. Accessed: Feb 2, 2021.